

Microfrontales I — Ejercicios

10 ejercicios de dificultad creciente (fácil → medio) sobre microfrontales, despliegue, Domain-Driven Design y agrupadores de módulos.

Basado en los apuntes de "Microfrontales I". Puedes escribir tus respuestas en el espacio indicado bajo cada ejercicio.

Ejercicio 1 • Dificultad: Fácil

Define con tus propias palabras qué es un microfrontend (MFE) y explica a qué concepto del backend equivale.

Espacio para tu respuesta

Ejercicio 2 • Dificultad: Fácil

Indica si las siguientes afirmaciones son verdaderas o falsas, y corrige las que sean falsas:

- a) Un MFE debe compartir su fuente de datos con el resto de MFEs de la aplicación.
- b) Un MFE es autocontenido porque incluye su propia lógica de negocio.
- c) Un web component puede incluir libremente llamadas a una API de negocio.

Espacio para tu respuesta

Ejercicio 3 • Dificultad: Fácil

Propón dos ejemplos, distintos a los vistos en el documento, de piezas de una aplicación web que podrían convertirse en MFEs reutilizables, y justifica por qué en cada caso.

Espacio para tu respuesta

Ejercicio 4 • Dificultad: Fácil-Medio

Explica con tus palabras la diferencia entre desplegar un MFE en tiempo de compilación y desplegarlo en tiempo de ejecución. Indica al menos una ventaja y una desventaja de cada opción.

Espacio para tu respuesta

Ejercicio 5 · Dificultad: Fácil-Medio

Una empresa tiene una única aplicación monolítica en la que todos los desarrolladores trabajan sobre el mismo repositorio y sufren conflictos constantes de código. Describe los dos pasos de evolución arquitectónica que recomendarías seguir hasta llegar a una arquitectura de microfrontales, y qué mejora aporta cada paso.

Espacio para tu respuesta

Ejercicio 6 · Dificultad: Medio

Un banco quiere incorporar una nueva pasarela de pago. El proceso pasa por: un equipo de Arquitectura que evalúa proveedores, un equipo de Seguridad que audita la opción elegida, Dirección que aprueba el gasto y, finalmente, un equipo de Integración que conecta la pasarela con el resto de sistemas. Aplicando las señales de DDD que conoces, propón los dominios en los que dividirías este proceso.

Espacio para tu respuesta

Ejercicio 7 · Dificultad: Medio

Explica por qué, al modelar los dominios de "Cuentas" y "Tarjetas" de un banco, no es buena idea crear un único modelo de "Cliente" genérico que sirva para ambos. ¿Qué mecanismo debe usarse si un dominio necesita datos de otro?

Espacio para tu respuesta

Ejercicio 8 · Dificultad: Medio

Dado el siguiente fragmento de configuración de webpack, indica qué hace cada parámetro (entry, output, y la regla dentro de module.rules) y detecta si falta algo para que los archivos .scss se procesen correctamente:

```
module.exports = {
  entry: './src/index.js',
  output: { filename: 'bundle.js', path: __dirname + '/dist' },
  module: {
    rules: [
      { test: /\.scss$/, use: ['style-loader', 'css-loader'] }
    ]
  }
};
```

Espacio para tu respuesta

Ejercicio 9 · Dificultad: Medio

Tienes una aplicación con dos puntos de entrada: la app principal (./src/app.js) y un panel de administración (./src/admin.js). Escribe cómo definirías el entry y el output de webpack para que generen dos archivos de salida distintos, uno por cada punto de entrada, sin que se sobrescriban entre sí.

Espacio para tu respuesta

Ejercicio 10 · Dificultad: Medio

Caso integrador. Una aseguradora tiene un monolito con las secciones de pólizas, siniestros y facturación, mantenido por un único equipo que sufre constantes conflictos y despliegues lentos. Propón:

- a) Cómo dividirías la aplicación en MFEs aplicando DDD (qué dominios identificarías y con qué señal de DDD los detectas).
- b) Qué estrategia de despliegue (tiempo de compilación o tiempo de ejecución) recomendarías para cada MFE y por qué.
- c) Qué loaders necesitaría, como mínimo, la configuración de webpack de cada MFE si usan TypeScript y SCSS.

Espacio para tu respuesta
